Ders 1 Genişletilmiş Notu: Hesaplama Kuramına Genel Bakış, Küme Kuramı, Boole Mantığı ve Doğrudan ve Yapıcı İspat

Ders 1 Genişletilmiş Notu: Hesaplama Kuramına Genel Bakış, Küme Kuramı, Boole Mantığı ve Doğrudan ve Yapıcı İspat

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Konusu ve Kapsamı
- 2. Hesaplama Kuramının Soruları
- 3. Karmaşıklık Kuramı
- 4. Hesaplanabilirlik Kuramı
- 5. Otomat Kuramı
- 6. Küme Kuramı
- 7. Boole Mantığı
- 8. Teorem, İspat, Aksiyom, Hipotez, Lemma ve Sonuç
- 9. İspat Yazma İpuçları
- 10. Doğrudan İspat
- 11. Yapıcı İspat

1. Dersin Konusu ve Kapsamı

Bu ders Hesaplama Kuramı'dır (Theory of Computation), kodu BLM2502'dir. Hoca dersin büyük ölçüde otomatlar (automata) üzerine olacağını söyledi. Otomatlar makinelere benzer, ama teorik makinelerdir. Temel soru şudur: Bu teorik makineler neyi çözebilir, hangi problemlerin üstesinden gelebilir? Bu soruya bağlı olarak algoritma (algorithm) nedir, hesaplama (computation) nedir gibi sorular da işlenecektir. Hoca dersin ana temasının bu olduğunu söyledi.

Dersin tek kaynağı Michael Sipser'ın *Introduction to the Theory of Computation* kitabının üçüncü baskısıdır. Hoca bunu iyi bir kitap olarak nitelendirdi.

> **Hocanın vurgusu:** Kitabı okuyun, işlediğimiz ya da işleyeceğimiz konuların bütün alıştırma (exercise) ve problemlerini çözün. Dersi gerçekten iyi öğrenmek için bu gereklidir. Kitapta çözümlü alıştırma ve problemler de vardır, bir kısmının çözümü yoktur.

Hocanın verdiği konu listesi geçicidir (tentative):

- İspat türleri.
- Deterministik ve deterministik olmayan sonlu otomatlar (deterministic and nondeterministic finite automata).
- Bağlamdan bağımsız diller (context-free languages) ve yığın otomatları (pushdown automata).
- Church-Turing tezi (Church-Turing thesis), Turing makineleri (Turing machines) ve türleri.

- Zaman kalırsa NP-tamlık (NP-completeness), karar verilebilirlik (decidability) ve indirgenebilirlik (reducibility).

Hoca Turing makinelerinin kesinlikle işleneceğini söyledi. Ondan sonraki konuları "joker" ya da "bonus" konular olarak nitelendirdi: Zaman olursa işlenecek, olmazsa işlenmeyecek.

2. Hesaplama Kuramının Soruları

Hoca dersin yanıtlamaya çalışacağı soruları şöyle sıraladı:

- Bilgisayar donanımının ve yazılımının matematiksel özellikleri nelerdir?
- Hesaplama nedir?
- Algoritma nedir?
- Bilgisayarların sınırları nelerdir?

Sonuncu soru özellikle ilginçtir: Bilgisayarların yapamadığı bir hesaplama var mı? Hoca cevabı baştan söyledi: Evet, bilgisayarların yapamadığı hesaplamalar vardır. Bilgisayarlar çok çok güçlü olsa bile, bazı hesaplamaları teorik olarak yapamazlar. Hoca önce "sonsuz güçleri olsa bile" diye başladı, sonra "sonsuz demeyelim" diyerek ifadeyi "çok çok güçlü" diye değiştirdi.

Dersin genel amacı, gerçek dünyadaki bilgisayarları yansıtan biçimsel matematiksel hesaplama modelleri (formal mathematical models of computation) geliştirmektir. Böylece gerçek bilgisayarları kullanmadan, gerçek bilgisayarlar hakkında düşünebiliriz. Bunu da elde edeceğimiz modelleri ve araçları kullanarak matematiksel olarak yaparız.

Hoca bundan sonra üç alanı tanıttı: karmaşıklık kuramı, hesaplanabilirlik kuramı ve otomat kuramı.

3. Karmaşıklık Kuramı

Karmaşıklık kuramı (complexity theory) şu soruyu sorar: Bazı problemleri hesaplama açısından zor, bazılarını kolay yapan nedir? Buradaki "kolay" ve "zor", bilgisayarın bakış açısından söylenir.

3.1 Kolay ve Zor Problem

Kolay (easy) problem, bilgisayarın verimli (efficiently), yani hızlı bir şekilde çözebildiği problemdir. Zor (hard) problemi ise bilgisayar verimli çözemez. Bunun anlamı şudur: Problem, yani girdi büyüdükçe zorluk o kadar hızlı artar ki elinizdeki bilgisayar ne kadar güçlü olursa olsun bu problemleri hesaplamakta zorlanırsınız.

Hocanın kolay problem örnekleri:

- 1 milyon sayıyı sıralamak. Bunu çözen hızlı algoritmalar vardır.
- Bir telefon rehberinde ya da veri tabanında isim aramak.

Hocanın zor problem örneği, bütün dersler için ders programı hazırlamaktır (timetable scheduling). Elinizde dersler vardır, örneğin BLM2502 ve diğerleri. Zaman dilimleri vardır. Normalde sınıflar da vardır; hoca pandemi nedeniyle derslerin çevrimiçi yapıldığını, ama normalde sınıfların da hesaba katılacağını belirtti. Amaç, dersleri bu zaman dilimlerine ve sınıflara yerleştirmektir.

Burada girdi boyutu (input size) ders sayısıyla belirlenir. Ders sayısına n diyelim. n küçükken bu iş çok zor değildir. Ama n büyüdükçe, n sonsuza giderken, bilgisayarın problemi çözmek için ihtiyaç duyduğu süre de sonsuza gider. Hoca bu problemde sürenin üstel mi yoksa başka bir biçimde mi büyüdüğünden emin olmadığını açıkça söyledi.

3.2 Büyüme Hızı

Hoca süre-girdi grafiği üzerinden kolay ve zor ayrımını şöyle somutlaştırdı:

- **Doğrusal (linear) büyüme:** Girdi arttıkça gereken süre doğrusal artıyorsa problem kolaydır.
- **Polinom (polynomial) büyüme:** n^2 gibi bir grafik de kolay sayılır.

- **Üstel (exponential) büyüme:** Algoritma girdiye göre 2^n gibi üstel süre istiyorsa bu zor bir problemdir.

2^n gerçekten çok hızlı büyür. n arttıkça 2^n çok daha hızlı artar ve bilgisayarlar bunu makul sürede çözmekte zorlanır.

3.3 Neden Sınıflandırmak İstiyoruz?

Karmaşıklık kuramındaki merkezî soru şudur: Verilen bir problemi kolay ya da zor olarak nasıl sınıflandırabiliriz? Hoca "kolay" ve "zor" dedi, ama zamanımız olursa ikiden fazla sınıf göreceğimizi, örneğin "çok zor" gibi sınıflar olacağını ekledi.

Sınıflandırma önemlidir. Bir problemi kolay diye sınıflandırırsanız, onun için hızlı çalışan bir algoritma yazabileceğinizi bilirsiniz. Zor diye sınıflandırırsanız, makul sürede çözen bir algoritma bulamayacağınızı bilirsiniz. O zaman şunları yapabilirsiniz:

- Problemi değiştirmek.
- Probleme farklı bir açıdan bakmak, alternatifler aramak.
- Deterministik olarak çözmek yerine rastgele (random) çözmeye çalışmak.
- En iyi çözümü aramak yerine yeterince iyi (good enough) bir çözüm aramak.

Hangisini seçeceğiniz size kalmıştır, bunun üzerine ayrıca düşünürsünüz.

> **Hocanın vurgusu:** Asıl önemli nokta, problemi sınıflandırabilmektir. Sınıflandıramazsanız en iyi çözümü mü aramanız gerektiğinden, yoksa yeterince iyi bir çözümün mü gerektiğinden hiçbir zaman emin olamazsınız.

Burada sınıflandırmak, ispatlamak demektir. Bir problem alıp onun kolay ya da zor olduğunu ispatlarsınız. Bu bir tahmin, spekülasyon ya da basit bir mantık yürütme değildir.

Böyle ispatlar yapabilmek için bilgisayarın, makinenin, algoritmanın ve hesaplamanın kesin matematiksel tanımlarına ihtiyaç vardır. Hoca bunları bu derste öğreneceğimizi söyledi.

4. Hesaplanabilirlik Kuramı

1930'larda bilim insanları bazı temel matematiksel problemlerin bir bilgisayarla çözülemeyeceğini keşfetti. Hocanın ilginç bulduğu nokta şudur: O zamanlar bilgisayar yoktu. Bilgisayarlar 1940'larda icat edildi. Yani insanlar bilgisayarları icat etmeden önce, bilgisayarların bazı problemleri çözemeyeceğini biliyordu.

Hocanın verdiği örnek şu sorudur: Rastgele (arbitrary) bir matematiksel önerme doğru mu, yanlış mı? Bilgisayara bir önerme veriyorsunuz, bilgisayar da size önermenin doğru mu yanlış mı olduğunu söylemek zorunda. Bu problem çözülebilir değildir, yani bilgisayarlar bunu yapamaz.

> **Ek bilgi (derste anlatılmadı):** Hocanın sözünü ettiği sonuç büyük olasılıkla Church ve Turing'in 1936 dolaylarındaki çalışmalarıdır. Bu soru literatürde karar problemi (Entscheidungsproblem) adıyla bilinir.

Hesaplanabilirlik kuramının (computability theory) merkezî sorusu yine bir sınıflandırmadır, ama bu kez problemleri çözülebilir (solvable) ve çözülemez (unsolvable) diye ayırmak isteriz. Bunun önemi açıktır: Bir problem teorik olarak çözülemezse onun üzerinde zaman harcamazsınız, onu düşünmek gereksizdir.

5. Otomat Kuramı

Otomat kuramı (automata theory), farklı hesaplama modellerinin (computation models) tanımlarını ve özelliklerini inceler. Hoca hesaplama modellerine ve kullanım alanlarına şu örnekleri verdi:

- **Sonlu otomatlar:** Deterministik sonlu otomat (DFA) ve deterministik olmayan sonlu otomat (NFA). Metin işlemede (text processing) ve derleyicilerde (compilers) kullanılır.
- **Bağlamdan bağımsız gramerler (context-free grammars):** Programlama dillerinde ve yapay zekâda kullanılır.

- **Turing makineleri:** Gerçek bilgisayarların basit ve soyut bir modelini verir, gerçek bilgisayarlar hakkında düşünmemize yardım eder.

  Hocanın vurgusu: Turing makineleri gerçekten önemlidir. Gerçek bilgisayarların çözebildiği problemleri Turing makineleri de çözebilir. Bir problem Turing makinesiyle çözülemiyorsa gerçek bir bilgisayarla da çözülemez. Hoca kendisi için bunun en önemli hesaplama modeli olduğunu söyledi.

Karmaşıklık kuramı da Turing makinelerine dayandırılabilir.

Hoca bu genel bakışı dersin işleniş sırasına bağladı: Alanları ters sırayla anlattı, ders ise otomat kuramıyla başlayacak, ardından hesaplanabilirliğe, sonra karmaşıklığa geçecektir. Dersin çoğu otomat kuramı olacaktır. Karmaşıklık ve hesaplanabilirlikten çok fazla şey işlenemeyebilir, hatta hiç işlenmeyebilir.

> **Ders sonrası not:** Altyazıda hoca sırayı söylerken ikinci alan için "computation theory" dedi. Bağlama göre doğrusu hesaplanabilirlik kuramıdır (computability theory).

6. Küme Kuramı

Hoca ikinci bölümü küme kuramına (set theory) ayırdı. Bu konuları muhtemelen bildiğimizi söyleyerek hızlı geçti ve dersin geri kalanında kullanacağımız önemli noktaların kısa bir özeti olduğunu belirtti.

Küme (set), iyi tanımlanmış (well-defined) nesnelerin bir topluluğudur.

6.1 Bilinen Kümeler

- $\mathbb{N}$: Doğal sayılar kümesi.
- $\mathbb{Z}$: Tam sayılar kümesi. Eksi sonsuzdan artı sonsuza uzanır.
- $\mathbb{Q}$: Rasyonel sayılar kümesi.
- $\mathbb{R}$: Reel sayılar kümesi.

Rasyonel sayılar, iki tam sayının bölümü olarak yazılabilen sayılardır:

$$\mathbb{Q} = \left\{ \frac{m}{n} \;\middle|\; m \in \mathbb{Z},\ n \in \mathbb{Z},\ n \neq 0 \right\}$$

Hoca yazım kuralını da belirtti: Küme tanımında koşulların arasına yalnızca virgül koyarsa bunu "ve" diye okuyabiliriz.

6.2 Alt Küme ve Kuvvet Kümesi

A'nın her elemanı aynı zamanda B'nin de elemanıysa A, B'nin alt kümesidir (subset) ve $A \subseteq B$ yazılır.

B'nin kuvvet kümesi (power set), B'nin bütün alt kümelerinin kümesidir. Bu kümenin elemanları kümelerin kendisidir:

$$\mathcal{P}(B) = \{A \mid A \subseteq B\}$$

Boş küme ve B'nin kendisi de B'nin alt kümeleri arasındadır. Yani $\emptyset \in \mathcal{P}(B)$ ve $B \in \mathcal{P}(B)$ olur. Hocanın $B = \{1, 2, 3\}$ örneği, derste bu konuyla ilgili sonradan gelen soruda çözüldü (bkz. 8.3).

6.3 Birleşim, Kesişim, Fark, Kartezyen Çarpım ve Tümleyen

Birleşim (union), kesişim (intersection) ve fark (difference) işlemleri bildiğimiz işlemlerdir:

$$A \cup B = \{x \mid x \in A \vee x \in B\}$$

$$A \cap B = \{x \mid x \in A \wedge x \in B\}$$

$$A - B = \{x \mid x \in A \wedge x \notin B\}$$

Birleşimde eleman ya A'da ya B'de olmalıdır. Hoca burada "ve" demeye başlayıp "veya" kullanmamız gerektiğini söyleyerek kendini düzeltti. Kesişimde eleman ikisinde birden olmalıdır. Farkta eleman A'dadır, ama B'de değildir.

Hoca Kartezyen çarpımın (Cartesian product) bizim için yeni olabileceğini söyledi. Burada elemanlar ikililerdir (pairs). İkilinin bir bileşeni A'dan, öbürü B'den gelir:

$$A \times B = \{(x, y) \mid x \in A,\ y \in B\}$$

A'nın tümleyeni (complement), A'da olmayan elemanların kümesidir. Fakat A'nın dışında sonsuz sayıda şey olabilir. Bu yüzden önce evrensel kümenin (universal set) ne olduğunu da tanımlamamız gerekir. Tümleyen, evrensel kümede olup A'da olmayan elemanlardır:

$$\overline{A} = \{x \in U \mid x \notin A\}$$

6.4 İkili Bağıntı

A ve B kümeleri üzerindeki bir ikili bağıntı (binary relation), $A \times B$'nin bir alt kümesidir. Hoca bunu iki küme çizerek gösterdi: A'dan bir eleman, B'den bir eleman seçip eşleştiririz. Bu ikililer Kartezyen çarpımda yer alır, seçilen ikililerin kümesi de bir ikili bağıntıdır.

6.5 Fonksiyon

A'dan B'ye bir fonksiyon (function), $f : A \to B$ biçiminde yazılır ve özel bir ikili bağıntıdır. Şart şudur: A'daki her a elemanı için R bağıntısında ilk bileşeni a olan **tam olarak bir** sıralı ikili (ordered pair) vardır. O ikili (a, b) ise $f(a) = b$ yazarız. Bunu iki şekilde okuyabiliriz:

- f, a'yı b'ye götürür (f maps a to b).
- a'nın f altındaki görüntüsü b'dir (image of a under f is b). Hoca bu ifadeyi hiç kullanmadığını söyledi.

Soru: Tanımda ikincil bileşenin B'de olduğunu söylememiz gerekmiyor muydu? **Cevap:** Bağıntı zaten A'dan B'ye tanımlandığı için ikinci bileşen B'de olmak zorundadır, hoca bu nedenle bunu ayrıca yazmadı.

Hoca iki kümeli çizimle şartın ne dediğini açıkladı: A'daki her noktaya karşılık tam olarak bir nokta vardır. Bir eleman iki değere gidemez:

- $a \mapsto b$ ve $c \mapsto b$ durumu serbesttir. Farklı elemanlar aynı değere gidebilir.
- $a \mapsto b$ ve $a \mapsto b'$ durumu ($b \neq b'$) yasaktır. Aynı a iki farklı değere gidemez.

A kümesine f'nin tanım kümesi (domain), B kümesine ise hocanın adlandırmasıyla f'nin "range"i denir.

> **Ders sonrası not:** Hoca B'ye "range" dedi. Bu kullanım korunmuştur, ama B için değer kümesi (codomain) terimi de kullanılır. Bu ayrımda range ya da görüntü kümesi (image), f'nin fiilen aldığı değerlerin kümesidir ve B'nin bir alt kümesidir.

6.6 Denklik Bağıntısı

Bir denklik bağıntısı (equivalence relation), $R \subseteq A \times A$ biçiminde, yani A'nın kendisiyle Kartezyen çarpımının alt kümesi olan bir ikili bağıntıdır ve şu üç şartı sağlar:

- **Yansıma (reflexive):** Her $a \in A$ için $(a, a) \in R$.
- **Simetri (symmetric):** Her $a, b \in A$ için $(a, b) \in R$ ise $(b, a) \in R$.
- **Geçişme (transitive):** Her $a, b, c \in A$ için $(a, b) \in R$ ve $(b, c) \in R$ ise $(a, c) \in R$.

Hoca simetri şartını önce "$a \in A$ ve $b \in B$" diye söyledi, sonra burada B kümesi olmadığını belirterek kendini düzeltti: İki bileşen de A'dadır.

7. Boole Mantığı

Boole mantığında (Boolean logic) yalnız iki değer vardır: yanlış (false) ve doğru (true). Bunları çoğu zaman 0 ve 1 ile yazarız. 0 yanlış, 1 doğru demektir. Bazen yanlış için F, doğru için T de görülür.

7.1 Temel İşlemler

- **Değil (negation, NOT):** $\neg$ ile gösterilir.
- **Ve (conjunction, AND):** Ters V biçimindeki $\wedge$ ile gösterilir.
- **Veya (disjunction, OR):** V biçimindeki $\vee$ ile gösterilir.
- **Özel veya (exclusive or, XOR):** Daire içinde artı işareti $\oplus$ ile gösterilir.
- **Denklik (equivalence):** Çift yönlü ince ya da kalın okla, $\leftrightarrow$ ya da $\Leftrightarrow$ ile gösterilir.
- **Gerektirme (implication):** Tek yönlü okla, $\rightarrow$ ya da $\Rightarrow$ ile gösterilir.

  Ders sonrası not: Hoca denklik işleminin özel bir adı olmadığını söyledi. Bu işlemin adı çift yönlü koşuldur (biconditional), "ancak ve ancak" diye okunur. Mantık devrelerinde XNOR olarak bilinir.

7.2 Doğruluk Tabloları

Doğruluk tabloları (truth tables), işlemlerin bütün girdi durumları için ne değer verdiğini gösterir. Değil işlemi için $\neg 0 = 1$ ve $\neg 1 = 0$ olur. İki argümanlı işlemlerde dört olasılık vardır. Hoca her işlemi ayrı ayrı yazdı, burada tek tabloda birleştirildi. Tablo python ile tam taranarak doğrulandı.

P	Q	$P \wedge Q$	$P \vee Q$	$P \oplus Q$	$P \Leftrightarrow Q$	$P \Rightarrow Q$
0	0	0	0	0	1	1
0	1	0	1	1	0	1
1	0	0	1	1	0	0
1	1	1	1	0	1	1

Hocanın okuma kuralları şöyledir:

- **Ve:** Yalnız ikisi de doğruysa sonuç doğrudur. Kalan üç satır 0'dır.
- **Veya:** Yalnız $0 \vee 0$ yanlıştır. Kalan üç satır 1'dir.
- **XOR:** Argümanlar aynıysa, yani ikisi de yanlış ya da ikisi de doğruysa sonuç 0'dır. Farklıysa sonuç 1'dir.
- **Denklik:** XOR'un tersi gibidir. Argümanlar aynıysa doğru, farklıysa yanlıştır. Farklıysa argümanlar mantıksal olarak denk değildir.
- **Gerektirme:** Yalnız $P = 1$, $Q = 0$ satırında yanlıştır.

7.3 Gerektirmeyi Anlamak: Çöp ve Para

Gerektirme bir "eğer ... ise, o zaman ..." (if-then) cümlesi gibidir. İlk kısma öncül (antecedent) denir; hoca bu kelimeyi hiç kullanmadığını söyledi ve koşul (condition) da denebileceğini belirtti. İkinci kısma sonuç (consequence) ya da söz (promise) denir. Yani bir koşul ve bir söz vardır.

Hocanın sık görülen örneği şudur: P = "çöpü çıkarırsın", Q = "para alırsın". $P \Rightarrow Q$'nun değerine dört satırda bakalım:

- $P = 1$, $Q = 0$**:** Çöpü çıkardın ama para almadın. Gerektirme yanlıştır. Tablodaki tek 0 budur.
- $P = 1$, $Q = 1$**:** Çöpü çıkardın, parayı aldın. Gerektirme doğrudur. Bu durum kolaydır.
- $P = 0$, $Q = 0$**:** Çöpü çıkarmadın, para da almadın. Mantıklıdır; koşulu yapmadın, sözü ya da ödülü de almadın. Gerektirme doğrudur.

- $P = 0$, $Q = 1$**:** İlginç olan durum budur. Çöpü çıkarmasan bile parayı alırsan gerektirme yine doğrudur. Hoca buradaki noktanın, ödülün ya da sonucun gerçekleşmesi olduğunu söyledi.

  Ek bilgi (derste anlatılmadı): Gerektirme, koşul sağlanmadığında ne olacağı hakkında söz vermez. Bu yüzden $P = 0$ iken söz bozulmuş sayılmaz. Tablodan görüldüğü gibi $P \Rightarrow Q$ ile $\neg P \vee Q$ aynı değerleri verir.

Hoca bu işlemleri ispat tekniklerinde kullanacağımızı söyleyerek konuyu kapattı.

8. Teorem, İspat, Aksiyom, Hipotez, Lemma ve Sonuç

8.1 Teorem ve İspat

Matematikte teorem (theorem), doğru olan bir önermedir. Bir şeye teorem diyebilmek için doğru olduğunu ispatlamak gerekir. İspat (proof), bir teoremin doğru olduğunu gösteren bir argüman oluşturan matematiksel önermeler dizisidir.

8.2 Aksiyom

Aksiyom (axiom), altta yatan matematiksel yapılar hakkında bir varsayımdır (assumption). Aksiyomlar ispatsız kabul edilir, onlar için ispat istemeyiz. Sayıları genellikle çok azdır.

> **Ders sonrası not:** Hoca ilk tanımda aksiyomu "ispatlayamadığınız bir şey" diye söyledi, sonra "ispatsız kabul ediyoruz, ispat aramıyoruz" dedi. İkinci ifade doğrudur: Aksiyom, ispatlanamaz olduğu için değil, başlangıç varsayımı olarak seçildiği için ispatsız kabul edilir.

Hocanın örneği olasılık kuramının (probability theory) aksiyomlarıdır:

- **Aksiyom 1:** Bir olayın olasılığı negatif olmayan bir reel sayıdır: $P(E) \in \mathbb{R}$ ve $P(E) \geq 0$.
- **Aksiyom 2:** Örnek uzayın (sample space) olasılığı 1'dir: $P(S) = 1$. Yani bütün olası seçeneklerin olasılıkları toplanınca 1 elde edilir.
- **Aksiyom 3:** Ayrık (disjoint) kümelerden oluşan sayılabilir herhangi bir dizi için birleşimin olasılığı, olasılıkların toplamıdır: $P\left(\bigcup_i E_i\right) = \sum_i P(E_i)$.

  Ders sonrası not: Hoca aksiyom 1'de, Türkçe tekrarında da, "sıfırdan büyük" dedi. Doğrusu $P(E) \geq 0$ şartıdır, yani "negatif olmayan", olasılığı 0 olan olaylar da kapsanır. Aksiyom 3'te hoca önce "bağımsız (independent)" dedi. Doğrusu ayrıktır; hoca Türkçe tekrarında da "birbirinden ayrık" dedi. Bağımsız olaylar için bu toplam kuralı geçerli değildir.

Hoca, olasılık ve istatistik dersi almayanların bu aksiyomları anlamak zorunda olmadığını, yalnız örnek olarak verdiğini söyledi. Asıl nokta sistemin nasıl kurulduğudur. Aksiyomları kabul ederiz. Sonra bu üç varsayımdan yola çıkıp ispatlayarak bir kural, sonra başka bir kural, sonra bir başkasını elde ederiz. Böylece kitaplar yazılacak kadar büyük olan olasılık kuramı ortaya çıkar. Örneğin $P(\overline{A}) = 1 - P(A)$ bir aksiyom değildir, aksiyomlar kullanılarak ispatlanır.

> **Ek bilgi (derste anlatılmadı):** Bu kuralın ispatı kısadır. A ile $\overline{A}$ ayrıktır ve birleşimleri S'dir. Aksiyom 3 ile $P(A) + P(\overline{A}) = P(S)$, aksiyom 2 ile de $P(S) = 1$ olur. Buradan $P(\overline{A}) = 1 - P(A)$ çıkar.

Hoca bu zinciri adım adım kurdu:

- Aksiyomları ispatlamazsınız, onları varsayarsınız. Aksiyomlar varsayım gereği doğrudur.
- Birinci kuralı aksiyomları kullanarak ispatlarsınız.
- İkinci kuralı aksiyomları ve birinci kuralı kullanarak ispatlarsınız. Birinci kuralı ispatladığınız için onun doğru olduğunu artık bilirsiniz.
- Üçüncü kuralı aksiyomları, birinci kuralı ve ikinci kuralı kullanarak ispatlarsınız.

"Kural" yerine "teorem" diyebiliriz: Teorem 1, Teorem 2 vb. Aksiyomlar ise sorgulamadan, matematiksel ispat istemeden kabul ettiğimiz ilk şeylerdir.

8.3 Araya Giren Soru: Kuvvet Kümesi

Aksiyomları anlatırken gelen bir soru üzerine hoca kuvvet kümesini Türkçe açıkladı. Türkçe adını o an hatırlayamadı, bir öğrenci "kuvvet kümesi" diye yazdı. Kuvvet kümesi, bir kümenin alt kümelerinden oluşan kümedir. Hoca $B = \{1, 2, 3\}$ örneğini aldı: Boş küme, yalnız 1'i içeren küme, yalnız 2'yi içeren küme, yalnız 3'ü içeren küme, 1 ve 2'yi içeren küme, 1 ve 3'ü içeren küme, 2 ve 3'ü içeren küme, 1, 2 ve 3'ü içeren küme:

$$\mathcal{P}(\{1, 2, 3\}) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$$

Ders sonrası not: Hoca sayarken $\{2, 3\}$'ü atladı ve $\{1, 2, 3\}$'ü iki kez söyledi. Doğru liste yukarıdadır. Tam tarama 8 alt küme verir, $2^3 = 8$.

8.4 Aksiyomlar Üzerine Bir Soru

Soru: Aksiyomların doğruluğunu kanıtlamadan kabul ediyoruz. Bunların doğruluğu ayrıca gösterilmeli değil mi? Yoksa her şey yerine oturduğu, işe yaradığı için mi doğru sayıyoruz?

Cevap: Hoca kendi düşüncesini söyledi: Evet, parçalar yapboz gibi yerli yerine oturuyor, bunların işe yaradığını görüyoruz. Ama bu, aksiyomların doğru olduğuna dair matematiksel bir ispat olmuyor.

Hoca ardından konuyu Türkçe açıkladı. Aksiyom, ispatına girişmeyeceğimiz varsayımlarımızdır. Hoca önce kalkülüsün aksiyomlarını örnek vermek istedi. Sonra farklı matematikçilerin farklı şeyler kullandığını söyleyerek olasılığa döndü.

Bir kurallar bütünü, bir teorem bütünü oluşturmak istediğinizde, örneğin olasılık kuramını kurarken, bir yerden başlamanız gerekir. İşte başladığınız o yere aksiyom diyoruz. Olasılıkta üç kuralı bir kenara koyar, bu kurallardan yola çıkarak bütün sistemi oluştururuz.

Hocanın benzetmesi **araç kutusudur**: Aksiyomları kullanarak ispatladığımız her teorem, araç kutumuza attığımız yeni bir araç olur ve yöntemlerimizi artırır. Üç aksiyomdan başlarız, ama ileri seviye bir konuda, örneğin istatistikte, bir kuralı ispatlarken artık aksiyomlarla uğraşmayız. Aksiyomlardan oraya kadar gelmek mümkündür, ama çok çok zor olur. Bu yüzden aksiyomlardan başlayıp ispatlaya ispatlaya oluşturduğumuz teorem kümesini kullanırız.

8.5 Hipotez

Hipotez (hypothesis), teorem hâline getirmek istediğiniz şeydir. Daha ileri araştırma için başlangıç noktası olan, ama teorem olabilmesi için ispat gerektiren bir varsayım (supposition) ya da açıklamadır. Hipotezi ispatlayabilirseniz o teorem olur.

8.6 Lemma ve Sonuç

Lemma (lemma) ve sonuç (corollary) da teoremdir, yani ispatlanmışlardır. Hoca önce şöyle ayırdı: Lemma daha önce ispatlanmış teoremlerdir, sonuç ise teoremlerin özel durumlarıdır.

Soru: Teoremde de lemmada da ispatladığımız şeyler var. Lemma ile teorem arasındaki fark nedir?

Cevap: Hoca soruyu haklı buldu ve ders sırasında internette baktı. Okuduğu tanımlara göre:

- **Lemma:** Başka doğru önermeleri ispatlamakta kullanılan doğru bir önermedir. Daha önemli bir sonucun ispatında kullanılan yardımcı bir gerçektir, daha az önemli bir teorem gibidir. Çoğu zaman bir grup lemma, daha büyük bir sonucu ispatlamak için kullanılır.
- **Teorem:** Önemli sonuçlara genellikle teorem denir.
- **Sonuç (corollary):** Bir teoremden kolayca çıkan kısa sonuçlardır. Teoremi ispatladıktan sonra bir iki adımda çıkardığınız ufak sonuçlara bu ad verilir.

Okuduğu kaynak bu kelimelerin kesin olmadığını ve bir şeye lemma, teorem ya da sonuç demenin düzenleme amacıyla yapılan bir tercih olduğunu da vurguluyordu. Lemma da teorem kadar doğrudur. Hoca bunu şöyle yorumladı: Matematikçilerin kitaplarında farklı terminolojiler görülebilir. Sonuçta üçü de matematiksel olarak ispatlanmıştır ve artık araç kutumuzda kullanabildiğimiz şeylerdir.

Bir öğrencinin özeti de şöyleydi: Lemma, teoremi ispatlamaya yardımcı olan daha küçük, ispatlanmış bir şeydir.

> **Hocanın vurgusu:** Gördüğünüz gibi en önemli şeylerden biri ispat yapmaktır. Aksiyomlardan başlayıp bir şeyler ispatlarsınız. Bu yüzden ispat tekniklerini konuşmamız gerekir.

9. İspat Yazma İpuçları

Teknikleri görmeden önce hoca bazı ipuçları verdi. Soru şudur: Matematiksel önermeleri nasıl ispatlayabiliriz?

> **Hocanın vurgusu:** İspatlanacak teoremi okuyup tamamen anlayın. En zor kısım budur.

Bunun için önermeyi kendi cümlelerinizle yeniden yazın. Bu, gerçekte neyin istendiğini anlamanıza yardım eder. Ayrıntılara dikkat edin. Teoremdeki her küçük şey, örneğin bir değişkenin $\mathbb{R}$'de olduğunu söyleyen ifade, genellikle önemlidir.

9.1 Birden Fazla İspat İçeren Teoremler

Bazen tek bir teorem aslında iki teoremdir, ispatlanacak birden fazla şey içerir. Hoca iki örnek verdi.

Birincisi "ancak ve ancak" (if and only if) içeren teoremlerdir. A özelliği ancak ve ancak B özelliği ise ($A \Leftrightarrow B$), iki gerektirmeyi ayrı ayrı ispatlamanız gerekir:

- $A \Rightarrow B$
- $B \Rightarrow A$

İkincisi küme eşitliğidir. $A = B$ olduğunu ispatlamak için şu iki alt küme ilişkisini ayrı ayrı ispatlarsınız. Sıra önemli değildir:

- $A \subseteq B$
- $B \subseteq A$

> **Hocanın vurgusu:** Teoremde "ancak ve ancak" ya da küme eşitliği varsa, her iki yönü de ispatlamak zorundasınız. Buna dikkat edin.

9.2 Diğer İpuçları

- Teoremi anlamak için birkaç basit durumu çözmeyi deneyin.
- Anladıktan sonra ispatı yazmaya çalışın.
- İlk seferde ispatlayamazsanız endişelenmeyin. İspat bulmak zaman alır.
- Bu aynı zamanda bir deneyim işidir. Birkaç ispat yaptıkça yenilerini daha kolay yaparsınız, pratik gerekir.

10. Doğrudan İspat

Doğrudan ispatta (direct proof) teoreme doğrudan yaklaşırız: Girdilerden, yani verilen önermelerden başlar, sonuca gideriz.

Teorem: n tek pozitif bir tam sayıysa n^2 de tektir.

Hoca "tek" kelimesinin (odd) Türkçesini de söyledi: tek tam sayı.

İspat:

- n tek pozitif bir tam sayı olduğundan bir k tam sayısı için $n = 2k + 1$ yazılabilir.
- n^2 üzerine düşündüğümüz için onu hesaplayalım: $n^2 = (2k + 1)^2 = 4k^2 + 4k + 1$.
- Amacımız "çift artı bir" biçimine ulaşmaktır. İlk iki terimden 2'yi dışarı alırız: $n^2 = 2(2k^2 + 2k) + 1$.
- $2k^2 + 2k$ bir tam sayıdır, dolayısıyla $2(2k^2 + 2k)$ çifttir.
- Çift artı bir tektir. O hâlde n^2 tektir. ■

Hoca, ispatı daha açık hâle getirmek için bir iki kez yeniden yazmak gerektiğini söyledi. Doğrudan ispat budur: n'den ve tek olmasının anlamından başlar, bazı işlemlerle sonuca ulaşırsınız.

> **Ders sonrası not:** Hoca k için "sıfırdan büyük" ve "pozitif tam sayı" dedi, $2k^2 + 2k$ için de "açıkça sıfırdan büyük" dedi. Oysa $n = 1$ için $k = 0$ olur ve $2k^2 + 2k = 0$ çıkar. Doğrusu $k \geq 0$ ve $2k^2 + 2k \geq 0$ yazmaktır. İspatın sonucu değişmez, çünkü 0 da çifttir. Hoca bir yerde "odd plus one" dedi, hemen ardından "even plus one" diye düzeltti; doğrusu ikincisidir. Python ile $(2k + 1)^2$ ile $2(2k^2 + 2k) + 1$ polinomlarının katsayılarının aynı olduğu $(4, 4, 1)$ ve 200000'den küçük bütün tek pozitif n için n^2'nin tek olduğu doğrulandı. Asıl kanıt yukarıdaki cebirdir.

11. Yapıcı İspat

Hocanın anlattığı ikinci teknik yapıcı ispattır (constructive proof, proof by construction). Genellikle bir teorem belirli türden bir nesnenin var olduğunu (exists) söylediğinde kullanılır. Yapmanız gereken, o nesneyi ya da böyle bir nesneyi kurmaktır.

Teorem: $p = q^r$ olarak yazılabilen rasyonel bir p sayısı vardır; burada q ve r'nin ikisi de irrasyoneldir.

11.1 Rasyonel ve İrrasyonel Sayı

Hoca teoremi anlamak için tanımları hatırlattı. Rasyonel sayı $\frac{m}{n}$ biçiminde yazılabilen sayıdır; m ve n tam sayıdır. İrrasyonel sayı ise bu kümede olmayan sayıdır. Örnek π'dir: $3{,}14\ldots$ diye gider, ama onu asla bir tam sayının başka bir tam sayıya bölümü olarak yazamazsınız.

> **Ders sonrası not:** Hoca bu hatırlatmada yalnız m ve n'nin tam sayı olduğunu söyledi. 6.1'deki tanımda olduğu gibi $n \neq 0$ şartı da gerekir.

Teoremin dediği şudur: q ve r irrasyonel, ama q^r rasyoneldir. Bunun olabileceğini göstermenin bir yolu örnek vermektir. Bir q ve bir r bulup q^r'nin rasyonel olduğunu gösterirseniz işiniz biter.

11.2 İspat

Hoca şu seçimi yaptı:

- $q = \sqrt{2}^{\sqrt{2}}$. Hoca bunun irrasyonel bir sayı olduğunu söyledi.
- $r = \sqrt{2}$.

O zaman:

$$q^r = \left(\sqrt{2}^{\sqrt{2}}\right)^{\sqrt{2}} = \sqrt{2}^{\sqrt{2}\cdot\sqrt{2}} = \sqrt{2}^{\,2} = 2$$

2 açıkça rasyonel bir sayıdır. Hocaya göre bir örnek kurduğumuz için ispat burada biter.

> **Ders sonrası not:** Bu ispatın eksik bir adımı vardır: Hoca $\sqrt{2}^{\sqrt{2}}$'nin irrasyonel olduğunu gerekçesiz söyledi. Bu iddia doğrudur, ama kolay değildir. Gelfond-Schneider teoremine göre, $a \neq 0, 1$ cebirsel ve b irrasyonel cebirsel ise a^b aşkındır (transcendental), dolayısıyla irrasyoneldir. Bu teoremi kullanmayan bilinen ispat ise iki durumla ilerler: $\sqrt{2}^{\sqrt{2}}$ rasyonelse $q = r = \sqrt{2}$ seçilir; irrasyonelse derste verilen seçim kullanılır. Bu ikinci yol teoremi ispatlar, ama hangi durumun geçerli olduğunu göstermediği için örneği açıkça kurmaz.

Hoca bir sonraki tekniğin olmayana ergi (proof by contradiction) olacağını söyledi ve onu haftaya bıraktı.

11.3 Ders Sonunda Bir Soru: q^r Neden 2?

Soru: q^r'nin neden 2 çıktığını anlayamadım. Ben önce q'yu hesaplayıp sonra üzeri r'ye almaya çalıştım.

Cevap: Hoca Türkçe açıkladı: Üslü sayının üssü alınırken üsler çarpım hâline gelir. Yani $\sqrt{2} \cdot \sqrt{2} = 2$ olur, $\sqrt{2}$'nin karesi de 2'dir. Hoca q'yu önceden hesaplamanın gerekmediğini "q'yu hesaplayamazsın ki" diye anlattı.

Öğrenci kendi yolunu da aktardı, birlikte adım adım gittiler. Bu yol da aynı sonuca götürür. Bütün sayıları 2'nin kuvveti olarak yazalım. Tabanda $\sqrt{2} = 2^{1/2}$, üste de $\sqrt{2} = 2^{1/2}$ vardır. Üsler çarpılır, $\frac{1}{2} = 2^{-1}$ olduğu için:

$$q = \left(2^{1/2}\right)^{2^{1/2}} = 2^{\frac{1}{2} \cdot 2^{1/2}} = 2^{2^{-1} \cdot 2^{1/2}} = 2^{2^{-1/2}}$$

Aynısını bir üst kat için yapalım:

$$q^r = \left(2^{2^{-1/2}}\right)^{2^{1/2}} = 2^{2^{-1/2} \cdot 2^{1/2}} = 2^{2^0} = 2^1 = 2$$

Yani kuvvet 1 gelir ve sonuç yine 2 olur. Son eşitlikteki 2^0 yazımı için (ara adım eklendi).

> **Ders sonrası not:** Hoca "q'yu hesaplayamazsın ki" dedi. Bu sözün geçerli olduğu anlam, q'nun tam sayı ya da basit bir kesir olarak yazılamamasıdır. Sayısal olarak $q \approx 1{,}632527$ hesaplanabilir ve q^r kayan noktalı hesapta yuvarlama payıyla 2 çıkar. Kesin kanıt ise yukarıdaki üs cebiridir.

Dersin Toparlaması

Bu derste önce hesaplama kuramının sorularını ve üç alanını gördük. Karmaşıklık kuramı problemleri kolay ve zor diye, hesaplanabilirlik kuramı çözülebilir ve çözülemez diye sınıflandırır. Otomat kuramı ise bu sınıflandırmaları matematiksel olarak ispatlayabilmek için gereken hesaplama modellerini tanımlar. Derste bu alanlar ters sırada işlenecek, ağırlık otomat kuramında olacak ve Turing makinesi merkezî model olacaktır.

Ardından dersin dili olan küme kuramını ve Boole mantığını kısaca tekrar ettik. Kuvvet kümesi, Kartezyen çarpım, ikili bağıntı, fonksiyon, denklik bağıntısı ve gerektirme bundan sonra sık kullanılacak kavramlardır. Teorem, ispat ve aksiyom kavramlarıyla matematiksel sistemlerin aksiyomlardan başlayıp ispatlarla nasıl büyüdüğünü, ispat ipuçlarıyla da nasıl çalışılacağını gördük. İlk iki ispat tekniği doğrudan ispat ve yapıcı ispattır. Bir sonraki derste olmayana ergi ile devam edilecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.